Collaborative Document RSS day 3

2026-09-15-research-software-support-training

Welcome to The Workshop Collaborative Document.

This Document is synchronized as you type, so that everyone viewing this page sees the same text. This allows you to collaborate seamlessly on documents.

This is the Document for today: https://edu.nl/g6eqm

Collaborative Document day 1: https://edu.nl/nmwaj

Collaborative Document day 2: https://edu.nl/jnqpv

Collaborative Document day 3: https://edu.nl/g6eqm

Collaborative Document day 4: https://edu.nl/mwc48

Collaborative Document day 5: https://edu.nl/6jm6x

Code of Conduct

Participants are expected to follow these guidelines:

- Use welcoming and inclusive language.
- Be respectful of different viewpoints and experiences.
- Gracefully accept constructive criticism.
- Focus on what is best for the community.
- Show courtesy and respect towards other community members.

If you feel that the code of conduct is breached, please talk to one of the instructors (if the complaint is for one of the participants) or send an email to training@esciencecenter.nl (if the complaint is for one of the instructors).

Certificate of attendance

If you attend the full workshop we will send you a certificate of attendance.
We will delete all personal identifyable information 8 months after the training.

⚖ License

All content is publicly available under the Creative Commons Attribution License: creativecommons.org/licenses/by/4.0/.

Getting help

To ask a question, raise your hand in zoom. Click on the icon labeled “Reactions” in the toolbar on the bottom center of your screen,
then click the button ‘Raise Hand ’. For urgent questions, just unmute and speak up!

You can also ask questions or type ‘I need help’ in the chat window and helpers will try to help you.
Please note it is not necessary to monitor the chat - the helpers will make sure that relevant questions are addressed in a plenary way.
(By the way, off-topic questions will still be answered in the chat)

Workshop website & Lesson materials

RSST Sept-Oct 2026
https://esciencecenter-digital-skills.github.io/2026-09-15-ds-rss-masterclass/

RSST lesson material
https://esciencecenter-digital-skills.github.io/research-software-support/

Instructors

Jaro Camphuijsen
Ewan Cahen
Sander van Rijn

Helpers

Fenne Riemslagh
Eva Lekkerkerker

Agenda

Time Topic
09:00 Welcome and recap
09:15 Software Testing
10:15 Break
10:30 Documentation
11:45 Break
12:00 Software Citations
13:00 End of Day 3

Questions/homework from last week

- If you have time, please finish your SMP. https://esciencecenter-digital-skills.github.io/research-software-support/modules/softwaremanagementplans/smp_questionnaire. No need to hand them in (we are not checking you), but we suggest you go over the remaining points. We reserve a bit of time at the start of day 3 to go over any questions about SMPs.
- What are examples of semantic / calenderial versioning?
    - Examples from the Research Software Directory: https://research-software-directory.org/
    - Haddock (https://research-software-directory.org/software/haddock3) changed from semantic versioning to calendar versioning halfway through - v3.0.0-beta.5 --> 2026.8.0
    - GGIR (https://research-software-directory.org/software/ggir) uses a semantic

versioning in the convention Major.Minor-Patch
    - EC-EARTH 4 (https://ec-earth-4-docs.readthedocs.io/en/latest/getting_started/getting_started.html#tagged-version) uses semantic versioning in the convention Major.Minor.Patch

Collaborative Notes

Software Testing

Process of evaluating and verifying that a software and its components meet specified requirements and work as expected. Its primary purpose is to identify defects, errors, flaws or bugs early in the development process.

If you run software with an expected outcome in mind is a kind of test.

If you can prevent a problem in the design that's better. It will always pay itself back. If you have to fix a bug in a major package, it's costly. If you fix a bug when the feature is just being introduced, that's easier.

Why test software?

- Ensure continued reliability and reproducibility
- Boost developer confidence
- Detect problems (updated) dependencies early
- Simplify and manage code complexity
- Confirm correct installation and demonstrate use

Automated Testing

- Faster execution: run tests in seconds/minutes vs hours of manual testing
- Higher accuracy: eliminates human error, ensuring consistent results
- Early detection: identifies issues when they arise
- Scalability: handles large test suites without extra effort
- Cost efficiency: saves time and resources in the long run

Test types

- Smoke Tests: verify that the essential functions of the software work
- Unit Tests: assess individuals components for correctness
- Integration Tests: verify interactions between different parts of the software
- System Tests: evaluate the complete and integrated software system
- Regression Tests: ensure new changes don't disrupt existing functionality

Continuous Integration and Deployment (CI/CD)

Provides an automated loop of tests that the computer runs for you.

Example scenario: you have a script running in your software repository and the server automatically tests the software with every new commit.

If your service requires to be up and running for multiple users, than it becomes more useful to use CI/CD.

Take home messages

- Automated software testing is key for quality, reliability and reproducibility
- Unit tests verify individual components, integration tests check component interactions
- CI/CD streamlines development and enhances deployment efficiency

Software Documentation

Somehow documentation is considered as the most boring part, as something to tick-off, but I actually think it's quite a fun part to do. It can be very useful not only for the other people, but also for yourself.

What is Documentation?

- Provides context for your work: can be text or video for example.
- Explains what this is and how to use it
- Explains what has been done and why: bit of the history how its been developed, or choices that have been made.

Important information for users and others that collaborate with you on your software.

Why document software?

The main reason is to make your software reusable. If you just share the code, people can run into trouble, because there's always things to explain how to interact with the software.

- A user should be able to run your software in the same way as you do now
- A user should be abe to install your software
- A contributor should be able to add to, improve or fix code

Documentation types

- User documentation
    - What does software do? How can it be used?
- Developer documentation
    - How can your software be modified or extended? Wy were certain development choices made
- Deployment documentation
    - How to install software? What hardware or other software is needed?

Documentation Formats

- README: simple text file, first thing that users/collaboraters see
- In-code: intended for developers (comments, docstrings, ...)
- Websites, Wikis, Youtube video
- Tutorials

A Good README file

- Which documents are where
- License
- Who the developers are, how to contact them
- What the application can do, and what the purpose is
- How to install and run it

- Versioning conventions - information for the contributors
- How it is maintained - the team, how often releases are made, how long people have to wait for their bug to be fixed, whom to contact
- Funding (if its part of a grant)
- Links to data, publications
- Citation information
    - It can be a separate file - we will talk more about citation later today

In-code documentation

- Code comments:
    - Provide context for (complex parts) of the code
    - Explains certain coding decisions
    - Can often be replaced by better programming
        - Make variable and function names explicit. Instead of T use Time.
- Docstrings:
    - Special comments that describe (use of) functions/classes/ver.
    - Can be called by users/websites
    - Follow a standardized syntax (per programming language)
    - Can be used to generate documentation pages
        - We try to add docstrings to all of our code.

User/API documentation

- What if a README file is not enough?
- Use tools to create API documentation from existing docstrings

What if your software has a number of unique functions and variables. You can create a list of these functions for users. Creatig docstrings allows you to automate generating a documentation.

Tutorials

- Example code that allows users to run specific test cases
- Jupyter notebooks are often used for this purpose

For users this is arguably the best way to get to know the software. It lets them see a demonstration of it. Notebooks (in python but also other programming languages) are a conventional way of presenting a tutorial.

Take-home message

- Documentation is a vital part of a project
- It should be kept and created alongside the corresponding code
- Depending on the purpose/scope documentation needs to meet different criteria
- Documentation can take many shapes

Software Citations

Why would you cite software

- Understanding Research Fields
- Credit for the author
- Recognition of the software
- Discovering software
- Reproducibilty

The software citation principles

- Importance
- Credit and attribution
- Unique Identification
- Persistence
- Accessibility
- Specificity

CFF File

You can add a CFF file to your GitHub/GitLab repository. In the example, you can see GitHub automatically present citation information as a button on the summary (sidebar information) of your repository.

More information about CFF file:
https://citation-file-format.github.io/

Testing CFF tool

https://esciencecenter-digital-skills.github.io/research-software-support/modules/citation/create_citation_file

Excercises

Software Testing

1. Identify the Test Type

Exercise: https://esciencecenter-digital-skills.github.io/research-software-support/modules/testing/exercise

More information about testing types: https://esciencecenter-digital-skills.github.io/research-software-support/modules/testing/software-testing#types-of-testing

Answer:

- First paragraph: smoke test - a metaphore for the smoke test. Typical reaction to a smoke test: we hold production, we see where the issue is and fix it before continuing.
- Second paragraph: Unit test - the smallest components are checked individually.
  Integration test - multiple components working together tested.
  System test - the whole system (when everything is put together), is tested.
- Third paragraph: regression test (also associated with bug fixing) - repeat of testing when a new feature (color of the pen) is introduced

2. Self-test

https://esciencecenter-digital-skills.github.io/research-software-support/modules/testing/self-test

Question A: What is software testing?

1. A method for evaluating a software's compatibility with different operating systems.
2. The process of releasing updates and patches for software.
3. The process of verifying and validating that a software meets the required specifications and works as intended.
4. A step in the software lifecycle focused on community engagement and feedback.
5. The practice of documenting software functionalities.

Answer: The process of verifying and validating that a software meets the required specifications and works as intended.

Question B: Why is software testing important?

1. To build and maintain the community's trust by demonstrating commitment to quality.
2. To identify and fix bugs, ensure the software meets required standards, and assess its reliability and reproducibility.
3. To design the software's user interface and graphics.
4. To accelerate the overall software development process.
5. To facilitate easier collaboration among developers.

Answer: Multiple answers can be applicable! (Maybe all except 3 are valid)

- Software testing contributes to the community's trust in your software project
- To identify and fix bugs, ensure the software meets required standards, and assess its reliability and reproducibility.
- Conditional on automated testing: To accelerate the overall software development process: having confidence in your software's reliability, automated testing allows bug identification faster and accelerates the software development
- Conditional on automated testing: To facilitate easier collaboration among developers.

Question C: Which best describes Continuous Integration and its benefits for this project?

1. Continuous Integration is a practice where team members work in isolation to ensure their code is perfect before integrating it once at the end of the project, helping to avoid conflicts and errors.
2. Continuous Integration involves the use of specialized software to automatically translate code into multiple programming languages, making it more versatile and reducing workload.
3. Continuous Integration is a development practice where team members frequently integrate their work, often several times a day, to detect and fix integration issues early, thereby improving collaboration and code quality.
4. Continuous Integration is a method where team members manually review each other's code at the end of each day, ensuring that the code meets quality standards and is ready for client presentation.
5. Continuous Integration refers to the continuous updating of software to incorporate user feedback and new features, ensuring the software evolves to meet user needs.

Answer: 3. Continuous Integration is a development practice where team members frequently integrate their work, often several times a day, to detect and fix integration issues early, thereby improving collaboration and code quality.

This is the best answer. Some answers address other aspects of the process, but are not a definition.

Software Documentation

When to use which documentation?

https://esciencecenter-digital-skills.github.io/research-software-support/modules/documentation/exercise-levels

For the following scenarios, which forms of documentation would you suggest Joanne to incorporate in her project?

Scenario A: basic readme, in-code documentation, naming

Scenario B: in-code documentation, all above mentioned+ overview of components and user documentation

Scenario C: User documentation, a tutorial, naming, all above mentioned+tutorial+API reference

Software citation

Breakout room 1

we would cite packages that are scientific (not commercial), open access, and that they need it to continue existing (eg the academic recognition to get sponsorships). Also, if there are many alternatives then there is no reason to cite it.

- ubuntu: no
- word: no
- git: no
- numpy: yes
- ESMValTool: yes
- ClimAn: yes
- The original code on which you based your library ClimAn: depends on how much you depend on it. You should keep the list not too long.
- Your own local Python script, used to execute the analysis: no but it should be archived/accesible

Breakout room 2

- ubuntu: no
- word: no
- git: no
- NumPy: Yes mention, but only version
- ESMValTool: yes, this is very specific for the domain and for the research
- ClimAn (own library): yes, acknowledgement for own work, reproducibility
- Own python script: yes, for reproducibility

Breakout room 3

Ubuntu - No
Microsoft Word - No
Git - No
NumPy - Yes
ESMValTool - Yes
ClimAn -Yes
The original code on which you based your library ClimAn - Yes
Your own local Python script, used to execute the analysis - Yes

Which software would you cite and which not? Research software but not other software tools used in research. Why? Because versions matter more for research software and it is involved in procuding results.

Breakout room 4

- Ubuntu - no, no reason to cite something so broad
- MS Word - no, no reason to cite something so broad
- git - no, no reason to cite something so broad
- NumPy - yes, has a stated citation request to do so
- ESMValTool - yes, also has a request to do citation.
- ClimAn - seems like it's self-developed; depends. If this is the first time using this library created by oneself then it's more like research software, but if it's subsequent research after that research software was previously published then it makes sense to cite oneself in later publications

Breakout room 5

- Ubuntu: it is a common operative system, no need to be cited.
- word: no need, same reason as ubuntu.
- git : no, also widely used
- Numpy library: depend specifically in the type of paper/ research, it could be usefull to cite a specific version.
- ESMValTool: yes it is very specific for your research also mention the version.
- ClimAn: yes, definetely has to be cited.
- The original code: yes, sure, it needs to be cited.
- your own local python script: no need for citiation unless the journal is specific

Breakout room 6

- Numpy and above do not need to be cited, they are very common general tools (software used in research and not research software)
- ESMValTool and below need to be cited, they are very domain specific and should be considered research output
- Own script: also cite it to reference specific archived version used in your paper!

CFF Tool

https://esciencecenter-digital-skills.github.io/research-software-support/modules/citation/create_citation_file

Try out the CFF tool.

Questions from this week

Software Testing

Q: Is there a test that is better handled manually rather than automated?
A: User experience-type testing is better tested manually, but even then there is a lot of things that you can test automated; does the page respond on time? Smoke-tests would also be something to do manually regularly - it's not really aimed at specific parts. A lot of system-tests also double as smoke-test. Changing visual changes is also hard to test in an automated manner. However, always test automatically if you can. Everything you have to do manually takes longer, and is more mistake-prone.

Q: is evaluating a software compatability with operating system also a type of regression test?
A: Reserve the term "regresson testing" for changes that are happening in your own code. I think it falls under system test where you're checking how all components work together. Testing software compatability with operating system can definitely be part of testing. All the testing types are not necessarily mutually exclusive. If you run a unit test regularly, it can be a form of regression test.

Q: When providing support on testing (for instance emphasizing to researchers that it is an important aspect of software development when reviewing SMPs) I was wondering if there are useful resources I could refer (perhaps rather unexperienced coders) to?
A:

- For Python: Getting Started with Pytest (https://docs.pytest.org/en/stable/getting-started.html)
- For R: Testing Basics using Testthat (https://r-pkgs.org/testing-basics.html)
- We have more links on the content website (https://esciencecenter-digital-skills.github.io/research-software-support/modules/testing/reading)

Citation

Q: Is there a fixed list of keywords one can use for the CITATION.cff file?
A: For domain specific keywords, you could use the New ERC panel structure in 2021 and 2022 (https://erc.europa.eu/news/new-erc-panel-structure-2021-and-2022) for example (link to the PDF (https://erc.europa.eu/sites/default/files/document/file/ERC_Panel_structure_2021_2022.pdf)), which is used at the RSD to categorise projects. Another option is the NWO research fields (https://www.nwo.nl/en/nwo-research-fields).

Note that a large part of research software is domain agnostic, so the lists above are not applicable. In this case, think about technical keywords to describe the software, i.e. `GPU programming` or `distributed computing`.

:car: Parking lot

:book: Homework

Software Documentation

What is suitable documentation for "your" project?

https://esciencecenter-digital-skills.github.io/research-software-support/modules/documentation/ex_suitability

Resources

- bump-my-version (https://callowayproject.github.io/bump-my-version/): A tool to consistently update any reference to your version number when creating a new release. This can be in your CITATION.cff, README.md, code, etc.